

Building a browser-based AI image dataset curation tool

The most practical architecture combines three tiers: in-browser ONNX models (~20MB total) for offline face detection and embedding, a local Python/Ollama server for heavy inference and captioning, and cloud APIs via OpenRouter for high-quality vision-language tasks. This approach lets the tool work offline for core operations while scaling up when connectivity and compute allow. Below is a comprehensive technical reference covering every component you'll need.

1. Face recognition models: the 2024-2025 landscape

The open-source face recognition ecosystem has consolidated around **InsightFace's ONNX model zoo** as the de facto standard. ArcFace (Additive Angular Margin Loss, CVPR 2019) remains the backbone architecture, achieving **99.83% on LFW** and top NIST FRVT rankings. The key models worth knowing:

InsightFace model packs (all ONNX, auto-download via `pip install insightface`):

Pack	Detection	Recognition	Total Size	License
buffalo_1 (default)	SCRFD-10GF	ResNet50 @ WebFace600K	326MB	Non-commercial
buffalo_s	SCRFD-500MF	MobileFaceNet @ WebFace600K	159MB	Non-commercial
buffalo_sc	SCRFD-500MF	MobileFaceNet	16MB	Non-commercial
antelopev2	SCRFD-10GF	ResNet100 @ Glint360K	407MB	Non-commercial

buffalo_1 contains 5 ONNX models: `det_10g.onnx` (face detector), `w600k_r50.onnx` (512-d recognition embeddings), `1k3d68.onnx` (3D landmarks), `2d106det.onnx` (2D landmarks), and `genderage.onnx`. [GitHub](#) [Legasyss](#) The HuggingFace mirror at huggingface.co/immich-app/buffalo_1 hosts these files.

AdaFace (CVPR 2022 Oral) adapts margins based on image quality, [MSU](#) achieving state-of-the-art on IJB-B, IJB-C, IJB-S, and TinyFace [ADS](#) — particularly strong on **low-quality faces** where ArcFace degrades. [GitHub: github.com/mk-minchul/AdaFace](https://github.com/mk-minchul/AdaFace). No pre-built ONNX models widely available; requires PyTorch export.

ElasticFace (CVPRW 2022) uses random Gaussian margins during training, extending SOTA on 7 of 9 benchmarks. [ResearchGate](#) [GitHub](#): github.com/fdbtrs/ElasticFace. CC BY-NC-SA 4.0 license, [ResearchGate](#) [GitHub](#) PyTorch only.

DeepFace (`pip install deepface`) wraps 11 recognition backends (VGG-Face, FaceNet, FaceNet512, ArcFace, GhostFaceNet, etc.) and 10 detection backends under a single Python API. (PyPI) It cannot run in browser but has a built-in REST server.

What runs in the browser via ONNX Runtime Web

The critical constraint is model size and operator support. These models work well with `onnxruntime-web`:

- **SCRFD-500M** (~2.4MB) — face detection with 5 keypoints. Download: `github.com/yakhyo/face-reidentification/releases` (Yakhyo)
- **SCRFD-2.5G** (~3.1MB) — better accuracy, still browser-friendly (Yakhyo)
- **MobileFaceNet w600k_mbf** (~13MB) — 512-d face embeddings, trained on WebFace600K (Yakhyo) (GitHub)
- **UltraFace** (~1.5MB) — ultra-lightweight detection (Readthedocs) from `github.com/Linzaer/Ultra-Light-Fast-Generic-Face-Detector-1MB`
- **ArcFace ResNet100 int8** (~65MB) — quantized version at `huggingface.co/onnxmodelzoo/arcfaceresnet100-11-int8`

The **buffalo_sc pack (16MB total)** is the sweet spot for browser deployment — it bundles detection + recognition in a small footprint. (Medium) For the smallest possible setup, pair UltraFace (1.5MB detection) with MobileFaceNet (13MB embedding) for **~15MB total**.

Models that **require a server**: full buffalo_l (326MB), AdaFace (no ONNX exports), ElasticFace (PyTorch only), VGG-Face (580MB), and anything using FAISS/database-backed search.

Existing browser-ready projects include **face-api.js** (original abandoned; maintained fork at `github.com/vladmandic/face-api`), now superseded by the **Human** library (GitHub) at `github.com/vladmandic/human`) and **retinaface-js** (`github.com/ShirasawaSama/retinaface-js`). (GitHub)

InsightFace REST API (self-hostable)

InsightFace-REST by SthPhoenix (`github.com/SthPhoenix/InsightFace-REST`) is a production-ready Docker-deployable API with TensorRT optimization achieving **up to 820 fps on RTX 4090**. (GitHub)

```
bash

git clone https://github.com/SthPhoenix/InsightFace-REST.git
cd InsightFace-REST && bash deploy.sh
# API docs at http://localhost:18081/docs
```

Key endpoint — `POST /extract` accepts base64 images and returns bounding boxes, 512-d embeddings, gender, and age. Supports batch processing, model selection (glintr100, w600k_r50, w600k_mbf), and msgpack serialization for 2× reduced network traffic. [GitHub](#)

DeepFace as a local REST server

DeepFace ships a built-in Flask/Gunicorn REST server: [PyPI](#)

```
bash

# Option 1: CLI
pip install deepface && deepface serve

# Option 2: Docker (recommended)
docker run -p 5005:5000 serengil/deepface
```

Endpoints on `http://localhost:5005`: [PyPI](#)

Endpoint	Method	Function
<code>/verify</code>	POST	1:1 face comparison (returns distance + verified boolean)
<code>/represent</code>	POST	Extract 512-d embedding vector
<code>/analyze</code>	POST	Age, gender, emotion, race analysis
<code>/search</code>	POST	Search against registered face database

Accepts base64 images, URLs, or file paths. Configurable `model_name` (ArcFace, FaceNet512, etc.) and `detector_backend` (retinaface, yolo, mtcnn) per request. [PyPI](#)

Free cloud APIs for face similarity

Provider	Free Tier	URL
CompreFace (self-hosted)	Unlimited, Docker-based	github.com/exadel-inc/CompreFace Arya.ai
AWS Rekognition	\$200 credits for 6 months	aws.amazon.com/rekognition
Azure Face API	30K detections + 1K recognitions/month FaceOnLive	azure.microsoft.com/en-us/products/ai-services/ai-vision

Provider	Free Tier	URL
Face++	Free tier with rate limits	<code>faceplusplus.com</code>
Luxand.cloud	500 requests/month <code>Luxand</code>	<code>luxand.cloud</code>

2. Vision API providers: pricing, CORS, and browser compatibility

For image captioning and quality analysis from the browser, **CORS support is the deciding factor**. Most API providers block direct browser calls.

CORS support matrix — the critical table

Provider	CORS?	Browser-Direct?	Best Vision Model	Input Price/1M tokens
OpenRouter	✔ Yes	Yes	Routes to all (Gemini, Claude, GPT, Llama)	Pass-through + markup
Google Gemini	⚠ Partial	Maybe (with <code>?key=</code> auth)	Gemini 2.5 Flash	\$0.15 (lite) — \$0.30
Groq	⚠ Unconfirmed	Possibly	Llama 4 Scout 17B×16E	\$0.11
Together.ai	? Unknown	Unlikely	Llama 4 Maverick 17B×128E	\$0.27
Replicate	✗ No	No — proxy required	LLaVA-13b, Moondream2	~\$0.002-0.006/image
Fal.ai	✗ No	No — proxy required	Primarily generative models	Per-image pricing
WaveSpeed AI	? Unknown	Unlikely	"Any LLM Vision" via OpenRouter	Varies by model

OpenRouter — best choice for browser-based tools

OpenRouter (`openrouter.ai`) explicitly supports CORS and routes to **300+ models** including all major vision models. `NVIDIA Developer Forums` It uses the OpenAI-compatible API format `OpenRouter` `OpenRouter` and offers free models via the `openrouter/free` router. `OpenRouter`

javascript

```
const response = await fetch("https://openrouter.ai/api/v1/chat/completions", {
  method: "POST",
  headers: {
    "Authorization": `Bearer ${OPENROUTER_API_KEY}`,
    "Content-Type": "application/json",
    "HTTP-Referer": "https://your-app.com",
    "X-Title": "Dataset Curation Tool"
  },
  body: JSON.stringify({
    model: "google/gemini-2.5-flash",
    messages: [{
      role: "user",
      content: [
        { type: "text", text: "Describe this image. Assess quality (sharpness, light"
        { type: "image_url", image_url: { url: `data:image/jpeg;base64,${base64Data}`
      ]
    }]
  })
});
const caption = (await response.json()).choices[0].message.content;
```

Security note: API keys are exposed in client-side code. OpenRouter supports OAuth PKCE flow and per-user credit limits ([OpenRouter](#)) for production apps.

Google Gemini API — cheapest option

The Gemini REST API at (generativelanguage.googleapis.com) may work directly from browsers with (`?key=`) query parameter auth, though multiple developers report CORS inconsistencies. ([Google AI](#)) Current pricing (per million tokens):

Model	Input	Output	Notes
Gemini 2.5 Flash-Lite	\$0.075	\$0.30	Best budget option
Gemini 2.5 Flash	\$0.30	\$2.50	Current recommended
Gemini 2.0 Flash	\$0.10	\$0.40	Deprecated June 2026

An image (~1024×1024) consumes ~**1,290 tokens**, ([Google Cloud](#)) so captioning one image costs roughly **\$0.0001–\$0.0004** depending on model. Free tier available with rate limits, no credit card required. ([Costgoat](#))

javascript

```
const resp = await fetch(
  `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash-lite:gen
  {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      contents: [{
        parts: [
          { text: "Caption this image and rate quality 1-10." },
          { inline_data: { mime_type: "image/jpeg", data: base64ImageData } }
        ]
      }]
    })
  }
);
```

Groq — fastest inference

Groq supports vision via **Llama 4 Scout** ([Groq](#) 594 tokens/sec) and **Llama 3.2 90B Vision** at [api.groq.com/openai/v1/chat/completions](#). Free tier available. [Groq Community](#) Uses OpenAI-compatible format with `image_url` content type. [Groq](#) Pricing: **\$0.11/M input tokens** for Llama 4 Scout. [groq](#)

3. Local model runners: Ollama, LM Studio, and ONNX Runtime Web

Ollama — the most practical local option

Ollama exposes a REST API on **port 11434** [OneUptime](#) [Single Quote](#) with both native (`/api/`) and OpenAI-compatible (`/v1/`) endpoints. [ComputingForGeeks](#) CORS is enabled by setting the `OLLAMA_ORIGINS` environment variable:

bash

```
# macOS
launchctl setenv OLLAMA_ORIGINS "*"

# Linux (systemd)
# Add Environment="OLLAMA_ORIGINS=*" to /etc/systemd/system/ollama.service

# Windows: Set OLLAMA_ORIGINS=* as system environment variable

# Docker
docker run -d -p 11434:11434 -e OLLAMA_ORIGINS="*" ollama/ollama
```

Detection from browser: `fetch('http://localhost:11434/')` returns "Ollama is running" if active. `Objectgraph` Models can be pulled programmatically via `POST /api/pull {"model": "moondream"}.` `OneUptime`

Vision models ranked by capability (from community benchmarks):

Model	Params	Size	Best For
qwen2.5vl:7b	7B	~5GB	Best overall captioning accuracy <code>PhotoPrism</code>
gemma3:4b	4B	~3GB	Fast, good quality, Google model
minicpm-v	8B	~5GB	Best OCR (surpasses GPT-4o on OCRBench) <code>Ollama</code> <code>Ollama</code>
llama3.2-vision:11b	11B	~8GB	Meta's vision model
moondream	1.8B	~1.7GB	Fastest, runs anywhere, lower accuracy <code>Ollama</code>
granite3.2-vision:2b	2B	~1.5GB	Document understanding

Sending images to Ollama from browser JavaScript:

```
javascript
```

```

async function captionImage(base64Image) {
  const response = await fetch('http://localhost:11434/api/generate', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'gemma3:4b',
      prompt: 'Describe this image in detail for a dataset caption.',
      images: [base64Image], // raw base64, no data:image prefix
      stream: false
    })
  });
  return (await response.json()).response;
}

```

LM Studio local server

LM Studio exposes a server from the **Developer tab** (Toolify) on **port 1234** (GPT for Work) with three API formats: native REST (`/api/v1/`), OpenAI-compatible (`/v1/chat/completions`), and Anthropic-compatible (`/v1/messages`). CORS is enabled via **Developer tab → Settings → "Enable CORS"** (GPT for Work) (GPT for Work) or CLI: `lms server start --cors`. (LM Studio)

Supports vision models (LLaVA, Gemma3) with image input via standard OpenAI format.

(LM Studio)

ONNX Runtime Web — browser ML inference

Install via `npm install onnxruntime-web` (ONNX Runtime) or CDN: `cdn.jsdelivr.net/npm/onnxruntime-web/dist/ort.min.js`. Three backends available:

Backend	Speed	Coverage	Status
WASM	Near-native CPU, multi-threaded	100% ONNX ops	Stable, default (npm)
WebGPU	Up to 19× faster than WASM	Growing subset	Recommended (Chrome 113+)
WebGL	GPU-accelerated	Partial	Deprecated/maintenance (npm)

Pre-converted ONNX models on HuggingFace/GitHub:

Model	Task	Size	Link
SCRFD-500M	Face detection	2.4MB	github.com/yakhyo/face-reidentification/releases (Yakhyo)
SCRFD-2.5G	Face detection	3.1MB	Same repo (Yakhyo)
SCRFD-10G	Face detection	16.1MB	Same repo (Yakhyo)
w600k_mbf (MobileFaceNet)	Face embedding	13MB	Same repo (Yakhyo)
w600k_r50 (ArcFace R50)	Face embedding	166MB	Same repo (Yakhyo)
ArcFace R100	Face embedding	249MB	huggingface.co/onnxmodelzoo/arcfaceresnet100-8 (Hugging Face)
ArcFace R100 int8	Face embedding	65MB	huggingface.co/onnxmodelzoo/arcfaceresnet100-11-int8
UltraFace	Face detection	1.5MB	github.com/Linzaer/Ultra-Light-Fast-Generic-Face-Detector-1MB
YOLOv8n-face	Face detection	12MB	huggingface.co/deepghs/yolo-face (Hugging Face +2)
YOLOv11n-face	Face detection	~6MB	huggingface.co/AdamCodd/YOLOv11n-face-detection (Hugging Face)

Complete browser inference example:

```
javascript
```

```
import * as ort from 'onnxruntime-web';
ort.env.wasm.numThreads = navigator.hardwareConcurrency || 4;

async function loadModel(modelUrl) {
  // Cache-first loading
  const cache = await caches.open('ml-models');
  let response = await cache.match(modelUrl);
  if (!response) {
    response = await fetch(modelUrl);
    await cache.put(modelUrl, response.clone());
  }
  const buffer = await response.arrayBuffer();
  return ort.InferenceSession.create(buffer, {
    executionProviders: [navigator.gpu ? 'webgpu' : 'wasm']
  });
}

async function getFaceEmbedding(session, preprocessedImageData) {
  // ArcFace/MobileFaceNet: input [1, 3, 112, 112], normalized (pixel-127.5)/128.0
  const tensor = new ort.Tensor('float32', preprocessedImageData, [1, 3, 112, 112]);
  const results = await session.run({ input: tensor });
  return results.output.data; // 512-d embedding
}
```

4. Offline-capable HTML app architecture

Model caching strategy: Cache API beats IndexedDB

Google Chrome's team **explicitly recommends the Cache API** for storing ML model files over IndexedDB. (Chrome Developers) The Cache API stores raw Response objects with zero serialization overhead and supports streaming, while IndexedDB requires serialize/deserialize cycles that cause memory pressure with large files. (Chrome Developers)

javascript

```
// Service Worker: sw.js
const MODEL_CACHE = 'ml-models-v1';
const MODEL_FILES = [
  '/models/scrfd_500m.onnx',      // 2.4MB face detection
  '/models/w600k_mbf.onnx',      // 13MB face embedding
  '/models/nima_mobilenet.onnx'   // 14MB quality assessment
];

self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(MODEL_CACHE).then(cache => cache.addAll(MODEL_FILES))
  );
  self.skipWaiting();
});

self.addEventListener('fetch', event => {
  if (new URL(event.request.url).pathname.startsWith('/models/')) {
    event.respondWith(
      caches.match(event.request).then(cached => cached || fetch(event.request).then(
        caches.open(MODEL_CACHE).then(c => c.put(event.request, resp.clone()))
      ))
    );
  }
});
```

Storage limits: Chrome allows ~60% of available disk per origin. ([web.dev](#)) Firefox ~50%. ([SitePoint](#)) Safari ~1GB per origin. Always request persistent storage via `navigator.storage.persist()` to prevent eviction.

TensorFlow.js offline pattern

TF.js natively supports IndexedDB model caching ([TensorFlow](#)) with a cache-first pattern:

```
javascript
```

```

async function loadModel() {
  try {
    return await tf.loadLayersModel('indexeddb://face-model');
  } catch (e) {
    const model = await tf.loadLayersModel('/models/blazeface/model.json');
    await model.save('indexeddb://face-model');
    return model;
  }
}

```

BlazeFace (~400KB) is the smallest browser face detector via TF.js, returning bounding boxes and 6 keypoints. **MediaPipe Face Landmarker** (~2MB) provides 468 3D facial landmarks. Both work offline once cached.

Best ONNX models for a ~20–35MB offline bundle

The optimal browser model stack:

- **Face detection:** SCRFD-2.5G_KPS (3.2MB) — best accuracy-to-size ratio with 5 keypoints for alignment ([PromptLayer](#))
- **Face embedding:** MobileFaceNet w600k_mbf (13MB) — 512-d embeddings, trained on WebFace600K, sufficient accuracy for similarity grouping
- **Image quality:** NIMA with MobileNet backbone (~14MB, convert from Keras via [github.com/ideal0/image-quality-assessment](#)) ([GitHub](#)) outputs a distribution over scores 1–10. ([arXiv](#)) Alternatively, **BRISQUE** requires no model file at all — it's a traditional algorithm implementable in pure JavaScript (~200 lines) or via OpenCV.js

Total offline bundle: ~30MB — downloads once, cached permanently via Service Worker + Cache API.

For higher accuracy when a server is available, fall back to the full buffalo_l pack (326MB) via the Python companion script.

5. Python companion script: complete implementation

The auto-installing Flask server

This script checks for missing packages, installs them automatically, ([Iditect](#)) and starts a CORS-enabled Flask server with face analysis endpoints:

python

```
#!/usr/bin/env python3
"""AI Dataset Curation Tool - Python Companion Server"""
import sys, subprocess, importlib.util

REQUIRED = [
    ("flask", "flask"), ("flask_cors", "flask-cors"),
    ("cv2", "opencv-python"), ("numpy", "numpy"),
    ("PIL", "Pillow"), ("insightface", "insightface"),
    ("onnxruntime", "onnxruntime"),
]

def ensure_packages():
    missing = [pip for imp, pip in REQUIRED if importlib.util.find_spec(imp) is None]
    if missing:
        print(f"Installing: {' '.join(missing)}")
        subprocess.check_call([sys.executable, "-m", "pip", "install", "--quiet"] +
                               missing)

ensure_packages()

import os, io, base64, numpy as np, cv2
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app) # Allow all origins - safe for localhost-only binding

_analyzer = None
def get_analyzer():
    global _analyzer
    if _analyzer is None:
        from insightface.app import FaceAnalysis
        _analyzer = FaceAnalysis(name="buffalo_l", providers=["CPUExecutionProvider"])
        _analyzer.prepare(ctx_id=0, det_size=(640, 640))
    return _analyzer

def decode_image(b64):
    if "," in b64: b64 = b64.split(",")[1]
    return cv2.imdecode(np.frombuffer(base64.b64decode(b64), np.uint8), cv2.IMREAD_COLOR)

@app.route("/detect-faces", methods=["POST"])
def detect_faces():
    img = decode_image(request.json["image"])
    faces = get_analyzer().get(img)
```

```

    return jsonify({"faces": [{
        "bbox": {"x": int(f.bbox[0]), "y": int(f.bbox[1]),
            "w": int(f.bbox[2]-f.bbox[0]), "h": int(f.bbox[3]-f.bbox[1])},
        "confidence": float(f.det_score)
    } for f in faces]})

@app.route("/face-embedding", methods=["POST"])
def face_embedding():
    img = decode_image(request.json["image"])
    faces = get_analyzer().get(img)
    return jsonify({"embeddings": [
        {"embedding": f.normed_embedding.tolist(), "confidence": float(f.det_score)}
        for f in faces
    ], "dimension": 512})

@app.route("/compare-faces", methods=["POST"])
def compare_faces():
    data = request.json
    faces1 = get_analyzer().get(decode_image(data["image1"]))
    faces2 = get_analyzer().get(decode_image(data["image2"]))
    if not faces1 or not faces2:
        return jsonify({"error": "No face detected"}), 400
    sim = float(np.dot(faces1[0].normed_embedding, faces2[0].normed_embedding))
    return jsonify({"similarity": sim, "is_same_person": sim > 0.4})

if __name__ == "__main__":
    PORT = int(os.environ.get("PORT", 5000))
    print(f"Server running on http://localhost:{PORT}")
    app.run(host="127.0.0.1", port=PORT)

```

Windows .bat launcher

```
bat
```

```

@echo off
title AI Companion Server
echo =====
echo   AI Dataset Curation - Companion Server
echo =====

REM Find Python
set PYTHON_CMD=
where python >nul 2>&1 && (set PYTHON_CMD=python& goto :found)
where py >nul 2>&1 && (set PYTHON_CMD=py -3& goto :found)
if exist "%~dp0python\python.exe" (set "PYTHON_CMD=%~dp0python\python.exe"& goto :found)

echo [ERROR] Python not found! Install from https://www.python.org/downloads/
echo       Check "Add Python to PATH" during installation.
pause & exit /b 1

:found
echo [OK] Using: %PYTHON_CMD%
%PYTHON_CMD% -m pip install flask flask-cors opencv-python numpy Pillow insightface
echo [Launch] Starting server... Press Ctrl+C to stop.
%PYTHON_CMD% "%~dp0server.py"
pause

```

For zero-Python users, bundle Python's official **embeddable distribution** (`python-3.12.x-embed-amd64.zip`, ~15MB) in a `python/` subfolder alongside the script. Edit `python312._pth` to uncomment `import site`, install pip via `get-pip.py`, then the .bat points to `%~dp0python\python.exe`.

CORS details for local Flask

`CORS(app)` with `origins="*"` correctly handles requests from `file://` origins (which send `Origin: null`), `http://localhost`, and `http://127.0.0.1`. This is safe when the server binds only to `127.0.0.1` since it's unreachable from the network. JSON POST requests with `Content-Type: application/json` trigger preflight OPTIONS requests, `Toolshelf +2` which flask-cors handles automatically.

Recommended architecture: putting it all together

The three-tier approach maximizes flexibility:

Tier 1 — Browser-only (always available, offline): ONNX Runtime Web running SCRFD-2.5G (detection, 3.2MB) + MobileFaceNet (embedding, 13MB). Service Worker caches models via Cache API. [Chrome Developers](#) Sufficient for face grouping, duplicate detection, and basic similarity scoring. Computed embeddings stored in IndexedDB for persistence. [web.dev](#)

Tier 2 — Local server (when installed): Python companion script or Ollama. The browser detects availability via `fetch('http://localhost:5000/')` or `fetch('http://localhost:11434/')`. Provides full buffalo_1 accuracy (512-d ArcFace embeddings), vision-language captioning (qwen2.5vl:7b or gemma3:4b via Ollama), and heavy batch processing. DeepFace's built-in REST server (`deepface serve`) is the simplest alternative. [PyPI](#)

Tier 3 — Cloud APIs (when connected): OpenRouter as the single gateway for all vision models. One API, one key, CORS-enabled, routes to [OpenRouter](#) Gemini 2.5 Flash (\$0.30/M tokens), [OpenRouter](#) Claude, GPT-4o, or free models. For direct Gemini access at lowest cost, the REST endpoint with `?key=` parameter may work but test CORS carefully.

Browser-side detection logic:

javascript

```
async function detectBackends() {
  const backends = { browser: true };
  try {
    const r = await fetch('http://localhost:5000/');
    backends.python = (await r.json()).status === 'ok';
  } catch { backends.python = false; }
  try {
    const r = await fetch('http://localhost:11434/');
    backends.ollama = (await r.text()).includes('running');
  } catch { backends.ollama = false; }
  backends.cloud = !!localStorage.getItem('openrouter_key');
  return backends;
}
```

This architecture lets users start with zero setup (browser-only), progressively enhance with a local server (double-click the .bat file), and optionally add cloud captioning by entering an API key. Every tier degrades gracefully when unavailable.